

EDA

Exploratory Data Analysis

EDA (Exploratory Data Analysis) is the first and most important step where we examine and understand the dataset before making conclusions or reports.

It helps analysts answer:

- What is happening in the data?
- Are there patterns or trends?
- Is the data clean and reliable?
- What insights can we extract?



Importance of EDA in Data Science



Before



After

What Do We Do in EDA (for Data Analysis)?

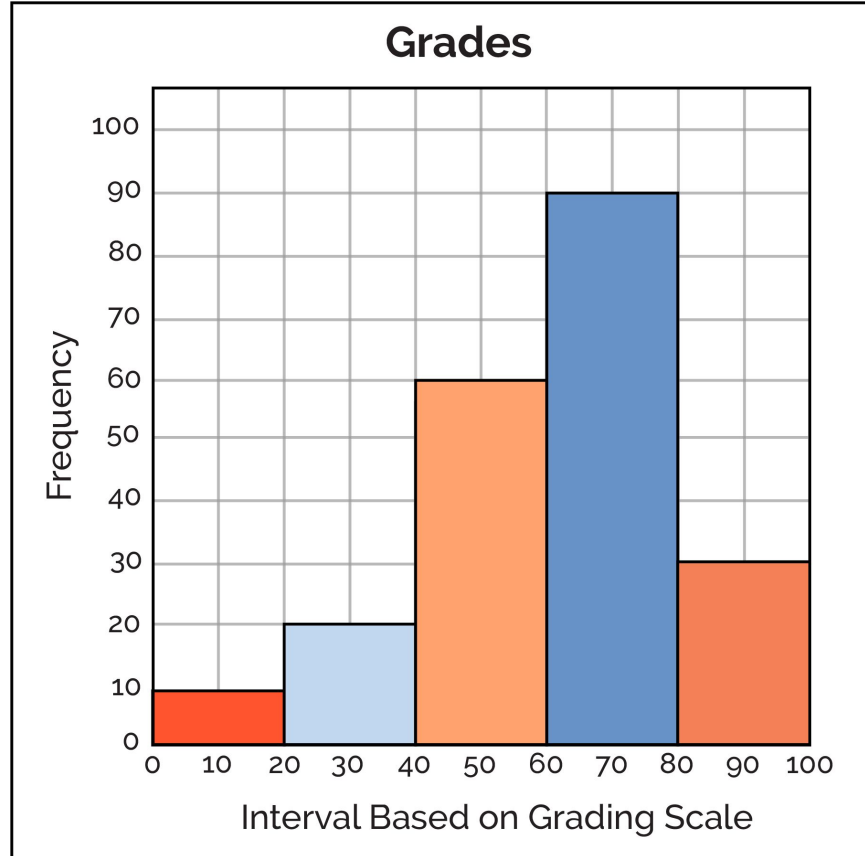
Understand the Dataset

- Number of rows & columns
- Data types (numeric, categorical)
- Summary statistics (mean, median, min, max)

Check Data Quality

- Missing values
- Duplicate records
- Incorrect data entries
- Outliers

Visualize the Data



Why is EDA Important in Data Analysis?

1. Helps Find Insights

You discover trends like:

- Sales increasing over time
- Which product sells most
- Which city generates highest revenue

2. Improves Decision Making

Managers make better business decisions based on patterns found.

3. Detects Problems Early

Wrong data → Wrong conclusions → Wrong decisions.

EDA prevents this.

4. Makes Reports More Meaningful

Instead of just numbers, you present clear visual insights.

EDA is the process of exploring and summarizing data using statistics and visualization to understand patterns, detect errors, and extract meaningful insights.

Overview of Important Python Libraries

Pandas

pandas is a Python library used for **data manipulation and analysis**.

pandas

```
df['State'].value_counts(normalize=True)
```

CA	0.121532
FL	0.093791
PA	0.076618
OH	0.046235
MO	0.042272
MI	0.036988
GA	0.034346
NY	0.033025
NC	0.030383
CO	0.025099

sidetable

```
df.stb.freq(['State'])
```

	State	Count	Percent	Cumulative Count	Cumulative Percent
0	CA	92	0.121532	92	0.121532
1	FL	71	0.093791	163	0.215324
2	PA	58	0.076618	221	0.291942
3	OH	35	0.046235	256	0.338177
4	MO	32	0.042272	288	0.380449
5	MI	28	0.036988	316	0.417437
6	GA	26	0.034346	342	0.451783
7	NY	25	0.033025	367	0.484808
8	NC	23	0.030383	390	0.515192
9	SC	19	0.025099	409	0.540291
10	MN	19	0.025099	428	0.565390
11	CO	19	0.025099	447	0.590489

NumPy

NumPy stands for *Numerical Python*.

It is used for **mathematical and numerical operations**.



Matplotlib

Matplotlib is used for **data visualization**.



Seaborn

Seaborn is built on top of Matplotlib and provides **beautiful statistical visualizations**.



seaborn

What are Dataset Formats?

Structured ways to store and organize data

Used for data storage, sharing, and analysis

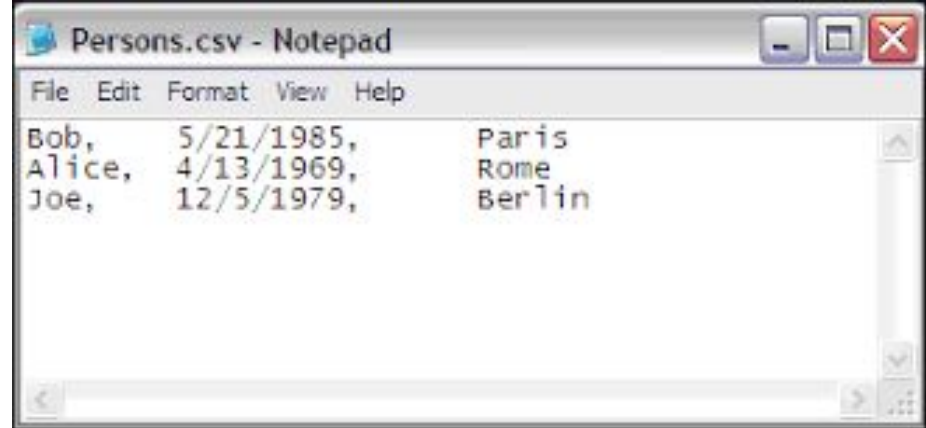
Common formats: CSV, Excel, JSON

Each format has different strengths and use cases

What is CSV?

CSV (Comma-Separated Values)

- Plain text file format
- Data separated by commas
- Each line = one row
- Simple and lightweight



Advantages & Disadvantages of CSV

Advantages:

- Easy to create and read
- Supported by almost all tools
- Lightweight and fast

Disadvantages:

- No formatting or styling
- Cannot store complex data structures
- No multiple sheets

What is Excel?

Excel Format (.xlsx)

- Developed by Microsoft
- Spreadsheet-based format
- Supports multiple sheets
- Allows formulas, charts, formatting

Used in:

- Business reporting
- Financial analysis
- Data management

Advantages & Disadvantages of Excel

Advantages:

- User-friendly interface
- Supports formulas and charts
- Can store multiple sheets

Disadvantages:

- Larger file size
- Not ideal for very large datasets
- Requires specific software

What is JSON?

JSON (JavaScript Object Notation)

- Text-based data format
- Stores data in key-value pairs
- Commonly used in web applications and APIs

```
{  
  "results": [  
    {  
      "bill_id": "s1917-113",  
      "chamber": "senate",  
      "congress": 113,  
      "number": 62,  
      "question": "On Passage of the Bill S. 1917",  
      "required": "1/2",  
      "result": "Bill Passed",  
      "roll_id": "s62-2814",  
      "roll_type": "On Passage of the Bill",  
    }  
  ]  
}
```

Advantages & Disadvantages of JSON

Advantages:

- Supports nested/complex data
- Lightweight
- Widely used in web development

Disadvantages:

- Harder to read for beginners
- Not ideal for tabular-only data

Introduction to Numpy Library

What is NumPy

NumPy = Numerical Python

It is a powerful Python library used for:

- Mathematical calculations
- Working with arrays
- Scientific computing
- Data analysis

👉 It is the foundation of many libraries like:

- Pandas
- Matplotlib
- Scikit-learn



Why Do We Need NumPy?

Normal Python lists:

- Slow for large data
- Cannot handle advanced math easily
- Not memory efficient

NumPy:

- Faster
- Uses less memory
- Supports mathematical operations easily

Why is NumPy Fast?

Written in C Language

Even though we use Python, NumPy works internally in C → Faster execution.

Uses Contiguous Memory

Data is stored in continuous memory blocks

→ Faster access

→ Better CPU performance

Less Memory Usage

NumPy arrays:

- Store same data type
- Use fixed memory size
- More efficient than Python lists

What is an Array in NumPy?

A NumPy array is a collection of elements stored in **continuous memory**.

It is called:

👉 `ndarray` (N-dimensional array)

We create arrays using different functions.

array()

Used to convert list → NumPy array

```
import numpy as np  
arr = np.array([1, 2, 3, 4])  
print(arr)
```

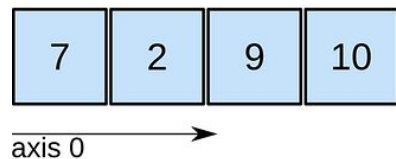
2D Array

```
arr = np.array([[1,2,3],  
                [4,5,6]])  
print(arr)
```

3D Array

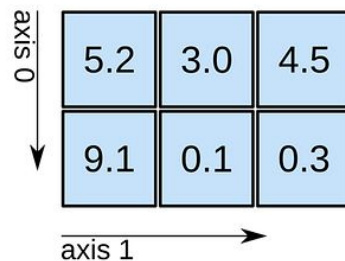
```
import numpy as np
arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(arr)
```

1D array



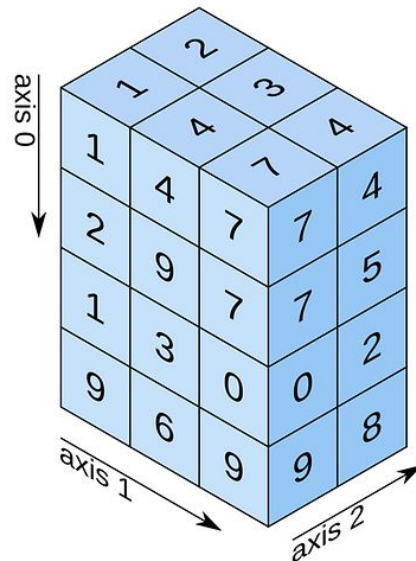
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

arange()

Used to create numbers in a sequence

Similar to Python `range( )` but returns NumPy array.

```
np.arange(start, stop, step)
```

```
np.arange(1, 10)
```

```
np.arange(0, 10, 2)
```

zeros()

Creates array filled with 0

```
np.zeros((rows, columns))
```

```
np.zeros((2,3))
```

Where zeros() is Used?

- ✓ Initializing weights in ML
- ✓ Creating empty data structures
- ✓ Placeholder arrays

ones()

Creates array filled with 1

```
np.ones((rows, columns))
```

```
np.ones((2,3))
```

```
[[1.  1.  1.]  
 [1.  1.  1.]
```

Where ones() is Used?

- ✓ Machine Learning initialization
- ✓ Testing calculations
- ✓ Creating sample datasets

Array Indexing, Slicing & Data Types in NumPy

	1	3	5	7	9
index	0	1	2	3	4
negative index	-5	-4	-3	-2	-1

What is Indexing?

Indexing means:

👉 Accessing a specific element from an array using its position.

Position always starts from **0** (zero-based indexing).

Indexing in 1D Array

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[0])    # First element
print(arr[3])    # Fourth element
```

Indexing in 2D Array

```
arr = np.array([[1,2,3],  
                [4,5,6]])  
  
print(arr[0,1])    # Row 0, Column 1
```

What is Slicing?

Extracting multiple elements from an array.

```
array[start:stop:step]
```

Slicing in 1D Array

```
arr = np.array([10,20,30,40,50])  
  
print(arr[1:4])
```

With Step

```
print(arr[0:5:2])
```

Slicing in 2D Array

```
arr = np.array([[1,2,3],  
                [4,5,6],  
                [7,8,9]])  
  
print(arr[0:2, 1:3])
```

```
array[row_start:row_end , column_start:column_end]
```

Data Types in NumPy

Every NumPy array has a fixed data type.

It tells us:

👉 What kind of values the array stores.

Data Type	Meaning
int32	Integer
float64	Decimal numbers
bool	True/False
str	String
complex	Complex numbers

Checking Data Type

```
arr = np.array([1,2,3])  
print(arr.dtype)
```

Changing Data Type

```
arr = np.array([1,2,3])  
new_arr = arr.astype('float')  
  
print(new_arr)  
print(new_arr.dtype)
```

Why Data Types Matter?

Memory efficiency

Faster calculations

Required in Machine Learning

Avoid errors during analysis

Array Reshaping and Dimensions

What is Array Dimension?

In **NumPy**, dimension means how many axes an array has.

Reshaping an Array

Reshaping means **changing structure without changing data**.

```
arr = np.array([1,2,3,4,5,6])  
  
new_arr = arr.reshape(2,3)  
print(new_arr)
```

⚠ Rule: Total elements must match

6 elements $\rightarrow$ can reshape into (2,3) or (3,2)

Concatenation (Joining Arrays)

Concatenate by Rows (axis=0)

```
a = np.array([[1,2],[3,4]])  
b = np.array([[5,6],[7,8]])  
  
result = np.concatenate((a,b), axis=0)  
print(result)
```

Concatenate by Columns (axis=1)

```
result = np.concatenate((a,b), axis=1)  
print(result)
```

Using vstack() and hstack()

```
np.vstack((a,b))    # Vertical stack  
np.hstack((a,b))    # Horizontal stack
```

Splitting Arrays

Split into Equal Parts

```
arr = np.array([1,2,3,4,5,6])  
new_arr = np.split(arr, 3)  
print(new_arr)
```

Split 2D Array (Row-wise)

```
arr = np.array([[1,2],[3,4],[5,6]])  
new_arr = np.split(arr, 3)
```

hsplit() and vsplit()

```
np.vsplit(arr, 3)    # Split vertically  
np.hsplit(arr, 2)    # Split horizontally
```

Array Filtering and Boolean Indexing

What is Array Filtering?

Array filtering means:

👉 Selecting only those elements that match a condition.

Example:

- Select numbers greater than 5
- Select even numbers
- Select negative values

What is Boolean Indexing

Boolean means **True or False**.

When we apply a condition to a NumPy array, it returns:

- **True** → condition satisfied
- **False** → condition not satisfied

Then NumPy selects only **True** values.

```
import numpy as np
```

```
arr = np.array([1, 4, 7, 10, 3, 8])
```

```
arr > 5
```

Use Boolean Indexing

```
filtered = arr[arr > 5]  
print(filtered)
```

Only values greater than 5 are selected.

Filter Even Numbers

```
arr[arr % 2 == 0]
```

Filter Negative Numbers

```
arr = np.array([3, -2, 7, -5, 10])
```

```
arr[arr < 0]
```

Always use **parentheses** with multiple conditions.

```
(arr > 3) & (arr < 8)
```

Sorting Arrays in NumPy

Sorting means arranging values in **ascending** or **descending** order.

```
import numpy as np

arr = np.array([5, 2, 9, 1, 7])

sorted_arr = np.sort(arr)
print(sorted_arr)
```

Sort in Descending Order

```
np.sort(arr)[::-1]
```

Sort by Columns

- `axis=0` → rows
- `axis=1` → columns

```
np.sort(arr, axis=0)
```

Searching in NumPy

Searching means finding positions or values.

```
arr = np.array([10, 20, 30, 40])  
  
index = np.where(arr == 30)  
print(index)
```

Search with Condition

```
np.where(arr > 20)
```

Basic statistical operations: mean(), median(), std(), sum()

```
arr = np.array([10, 20, 30, 40, 50])
```

Mean (Average)

```
np.mean(arr)
```

Median (Middle Value)

```
np.median(arr)
```

Standard Deviation (Spread of Data)

```
np.std(arr)
```

Sum (Total)

```
np.sum(arr)
```

With 2D Arrays

```
arr2d = np.array([[1, 2, 3],  
                  [4, 5, 6]])
```

Mean

```
np.mean(arr2d)          # overall  
np.mean(arr2d, axis=0)  # column-wise  
np.mean(arr2d, axis=1)  # row-wise
```

Sum

```
np.sum(arr2d, axis=0) # columns  
np.sum(arr2d, axis=1) # rows
```

Linear algebra: dot product, transpose

Dot Product (Matrix Multiplication)

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
  
np.dot(a, b)
```

$$(1 \times 4) + (2 \times 5) + (3 \times 6) = 4 + 10 + 18 = 32$$

Transpose (Row $\leftrightarrow$ Column)

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])  
  
A.T
```

Random number generation

Random Numbers (0 to 1)

```
np.random.rand(3)
```

2D Array

```
np.random.rand(2, 3)
```

Random Integers

```
np.random.randint(1, 10, size=5)
```

Pandas

Visualization Library

Data Visualization for Analytics & Storytelling

What is Data Visualization?

- Visual representation of data using charts and graphs.
- Why important:
 - • Humans understand visuals faster than tables
 - • Helps discover patterns
 - • Makes insights easy to explain

Why Data Visualization is Important in Analytics

- • Helps explore large datasets quickly
 - • Identifies trends and patterns
 - • Detects anomalies or outliers
 - • Supports business decision making
-
- Example: E-commerce companies track monthly sales trends.

Data Visualization in Storytelling

- Storytelling with data means explaining insights visually.
- Example:
- A chart showing customer growth after launching a mobile app clearly tells the success story.

Key Concepts in Visualization

- Important elements of any chart:
 - • Figure – the entire canvas
 - • Axes – horizontal and vertical scale
 - • Labels – explain data meaning
 - • Titles – describe what the chart shows

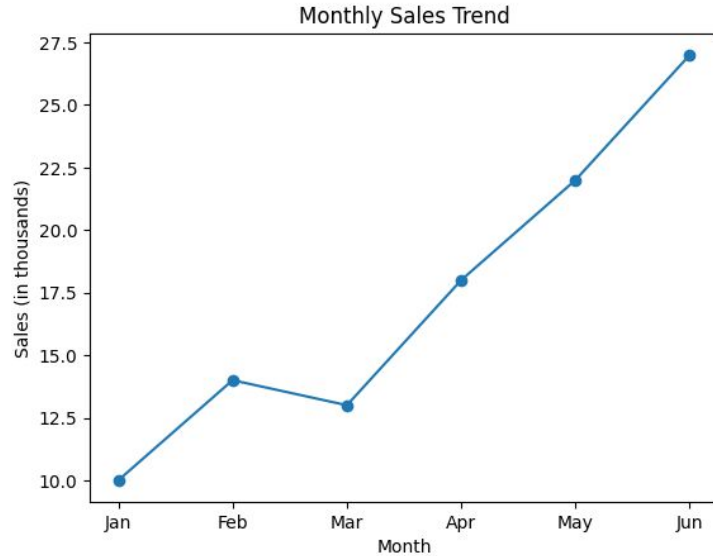
Figure and Axes

- Figure: The complete chart area.
- Axes:
 - X-Axis → categories or time
 - Y-Axis → numeric values
- Example:
 - X-axis: Months
 - Y-axis: Revenue

Labels and Titles

- Title explains what the chart represents.
- Axis Labels explain each axis.
- Example:
 - Title: Monthly Sales 2025
 - X-axis: Month
 - Y-axis: Sales (₹)

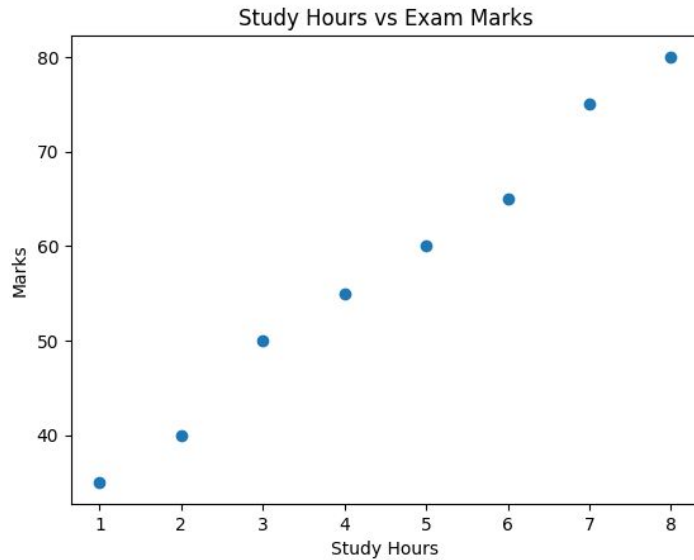
Line Plot – Showing Trends



Used to show trends over time.

Real Example:
Companies track revenue growth month by month.

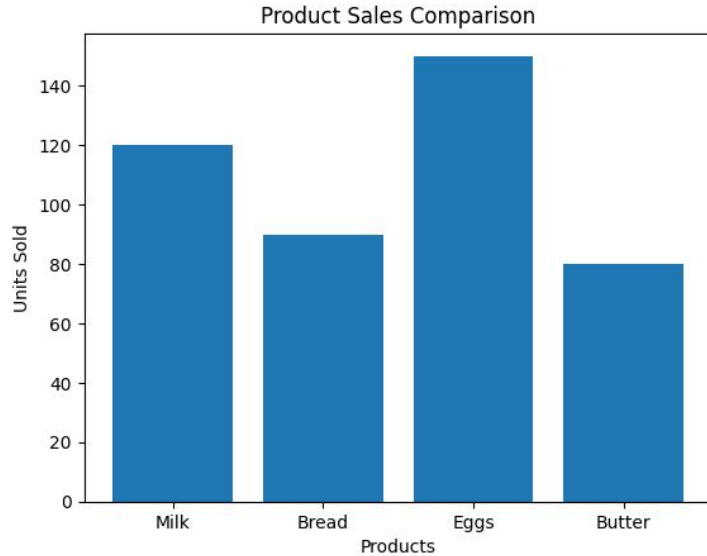
Scatter Plot – Showing Relationships



Shows relationship between two variables.

Example:
Study hours vs exam marks.

Bar Plot – Comparing Categories



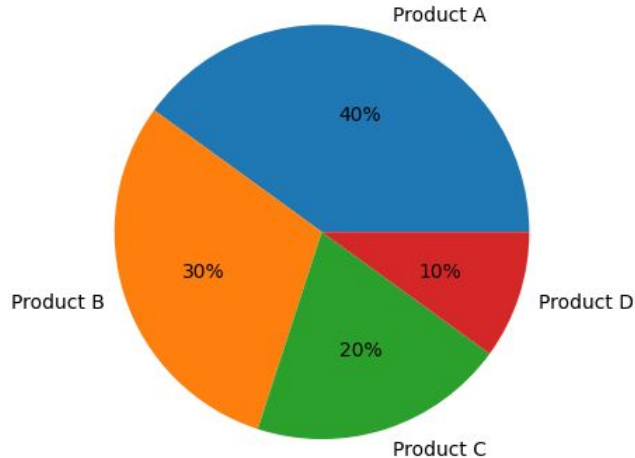
Best for comparison.

Example:

Comparing sales of different products.

Pie Chart – Showing Distribution

Revenue Contribution by Product



Used to show proportions or percentages.

Example:
Revenue contribution by product.

Visualization Goals

- Charts are used for different analytical goals:
 - • Comparison
 - • Trends
 - • Distribution
 - • Relationships

Examples of Visualization Goals

- Comparison → Product sales
- Trends → Monthly revenue growth
- Distribution → Exam score spread
- Relationship → Advertising spend vs sales

Key Takeaways

- • Visualization makes data easy to understand
- • Important for analytics and business decisions
- • Different charts solve different problems
- • Good visualization tells a clear story

